

Business Continuity & Disaster Recovery

Owner: Mark Stolte, Founder — markstolte02@gmail.com

Last reviewed: 2026-07-05

Stage: Pre-launch / active development

Purpose

Describes what keeps Penny Pilot's code, data, and ability to operate recoverable if a system fails, a vendor has an outage, or the founder is unavailable.

Infrastructure posture

Penny Pilot has no on-premises infrastructure to recover — the backend is entirely Supabase-managed (Postgres, Auth, Edge Functions) and the mobile app is built and distributed via Expo/EAS. This means most traditional "disaster recovery" concerns (server hardware failure, data-center loss) are inherited from those vendors' own DR programs rather than owned directly by Penny Pilot. What Penny Pilot does own:

- **Source code and schema:** version-controlled in GitHub ([Mstolte02/penny-pilot](#)), including every database migration ([supabase/migrations/](#)). The entire application and schema can be rebuilt from the repository against a fresh Supabase project if the original project were ever lost.
- **Application data:** lives in Supabase Postgres. Backup/point-in-time-recovery configuration depends on the Supabase plan tier in use — **action item:** confirm and document the specific backup/PITR window available on the current plan before onboarding real users, rather than assuming a default.
- **Local-only data:** in [mock](#) mode, or for a user who chooses the no-link CSV-import path without ever enabling Supabase sync, transaction data lives only in that device's [AsyncStorage](#). This data has no server-side backup by design — it is the same durability model as a spreadsheet kept on one device, and is disclosed to users as such rather than implied to be backed up.

Recovery scenarios

Scenario	Recovery approach
Supabase project data loss/corruption	Restore from Supabase's backup/PITR (once confirmed available on-plan); rebuild schema from supabase/migrations/ if starting from an empty project.

Supabase outage	No failover today — the app depends on Supabase being up. mock mode allows the app UI to still be demoed/developed during an outage, but it is not a live-user fallback.
Plaid outage	Bank sync pauses; the no-link CSV-import path (src/services/csv-import.ts) remains fully functional since it has no dependency on Plaid, giving users a way to keep using the app's budgeting features during a Plaid outage.
Loss of the GitHub repository	No secondary git remote exists today. Action item: add a periodic mirror/backup of the repository (e.g., to a second private remote) once the codebase represents meaningful, hard-to-recreate work.
Founder unavailable (illness, accident)	This is the most significant single point of failure today, since one person holds all vendor account access (see Access Control Policy). Action item: establish a documented, securely stored recovery record (e.g., a sealed credential-recovery kit with a trusted contact, or a password-manager emergency-access feature) so the business isn't unrecoverable if the founder is unreachable. Not yet in place as of this review.

Recovery objectives

Given the pre-launch stage and lack of a paid SLA today, no formal RTO/RPO is contractually promised to users. As aspirational internal targets:

- **RTO (time to restore service after an outage within Penny Pilot's control):** same day, best-effort.
- **RPO (acceptable data loss window):** whatever Supabase's backup/PITR interval provides once confirmed (see action item above) — targeting no more than 24 hours of loss for synced data.

These targets will be formalized (and likely tightened) before any paid tier ships with an implied or explicit SLA.

Testing

No formal disaster-recovery drill (e.g., restoring from a Supabase backup into a scratch project to confirm it works) has been performed yet. This is a planned action item before onboarding production users at scale, not something to assume works untested.